ArmaMCP Initial Setup and Activation Guide

ArmaMCP MVP

June 7, 2026

1 Purpose

ArmaMCP connects Codex to Arma 3 Eden Editor through a local Model Context Protocol (MCP) sidecar, a localhost HTTP bridge, a native Arma extension, and a HEMTT-built SQF addon.

```
Codex MCP client
  -> TypeScript stdio sidecar
  -> 127.0.0.1 HTTP bridge
  -> ArmaMCP_x64 native extension
  -> SQF Eden addon
```

The MVP activation path lets Codex ping the sidecar, request Eden selection snapshots, generate a dry-run checkpoint plan, queue a reviewed plan, and inspect bridge events from Eden.

2 Safety Model

ArmaMCP is local-first by design.

- The HTTP bridge binds to `127.0.0.1` by default.
- Bridge endpoints under `/bridge/*` require `Authorization: Bearer <ARMA_MCP_TOKEN>`.
- The sidecar token is supplied with the `ARMA_MCP_TOKEN` environment variable.
- The Arma extension reads the matching token from `ArmaMCP.ini` next to the loaded extension.
- No OpenAI keys or durable secrets should be placed in SQF, PBOs, mission files, extension source, compiled binaries, or committed config files.
- The MVP exposes no raw SQF execution tool, no shell execution, no remote execution, and no public multiplayer server control.
- Queued editor plans are restricted to structured `createObject` and `createMarker` operations.

3 Prerequisites

- Node.js and npm for the TypeScript sidecar.
- HEMTT on `PATH` for the Arma addon build.
- CMake and a C++17 compiler for the native Linux extension build.
- `x86_64-w64-mingw32-g++` if building the Windows/Proton DLL locally.
- Arma 3 installed. The local Arma 3 Tools path assumed by the project is:

```
/mnt/game_one/SteamLibrary/steamapps/common/Arma 3 Tools/
```

4 Repository Layout

The relevant project paths are:

- `/path/to/arma-mcp`: repository root.
- `sidecar/`: TypeScript MCP server and HTTP bridge.
- `addons/main/`: HEMTT addon and Eden SQF functions.
- `extension/`: native `ArmaMCP_x64` extension source.
- `scripts/build-extension.sh`: native extension build helper.
- `scripts/validate.sh`: full local validation helper.

5 Build The Sidecar

From the repository root:

```
cd /path/to/arma-mcp/sidecar
npm install
npm run build
```

To run the sidecar tests:

```
cd /path/to/arma-mcp/sidecar
npm run validate
```

The test suite opens temporary `127.0.0.1` ports. If a sandbox blocks local binding, the HTTP tests can fail with `listen EPERM`; run them in an environment that permits loopback binding.

6 Choose A Local Bridge Token

Pick a local token and keep it out of git. For example:

```
export ARMA_MCP_TOKEN='replace-with-a-local-random-token'
```

Use the same token for:

- the sidecar process;
- the Codex MCP server configuration;
- the local mod folder's `ArmaMCP.ini`.

For quick local-only smoke tests, documentation examples use `dev-token`, but that value is only a placeholder.

7 Configure Codex MCP

First build the sidecar:

```
cd /path/to/arma-mcp/sidecar
npm run build
```

Add this server entry to Codex `config.toml`, replacing the token with your local value:

```
[mcp_servers.arma-mcp]
command = "node"
args = ["/path/to/arma-mcp/sidecar/dist/index.js"]
env = { ARMA_MCP_TOKEN = "replace-with-a-local-random-token" }
startup_timeout_sec = 10
tool_timeout_sec = 60
```

If Arma is already polling a separately started bridge, configure Codex to reuse that bridge instead of binding the HTTP listener again:

```
[mcp_servers.arma-mcp]
command = "node"
args = ["/path/to/arma-mcp/sidecar/dist/index.js"]
env = {
  ARMA_MCP_TOKEN = "replace-with-a-local-random-token",
  ARMA_MCP_SKIP_HTTP_LISTEN = "1"
}
startup_timeout_sec = 10
tool_timeout_sec = 60
```

Alternatively, register it with the Codex CLI:

```
codex mcp add arma-mcp \
  --env ARMA_MCP_TOKEN=replace-with-a-local-random-token \
  -- node /path/to/arma-mcp/sidecar/dist/index.js
```

Reuse-mode helper form:

```
codex mcp add arma-mcp \
  --env ARMA_MCP_TOKEN=replace-with-a-local-random-token \
  --env ARMA_MCP_SKIP_HTTP_LISTEN=1 \
  -- node /path/to/arma-mcp/sidecar/dist/index.js
```

After starting Codex, inspect MCP servers with:

```
/mcp
```

Expected result: `arma-mcp` is listed and its tools are available.

8 Run The Bridge Without Codex

For bridge-only development:

```
cd /path/to/arma-mcp/sidecar
ARMA_MCP_TOKEN=dev-token npm run dev:http
```

This starts only the HTTP bridge. Normal MCP mode should be launched by Codex and keeps MCP traffic on stdout while logs go to stderr.

For local development of a Codex stdio process that reuses the running bridge:

```
ARMA_MCP_TOKEN=dev-token npm run dev:stdio-existing
```

9 Build The Native Extension

From the repository root:

```
cd /path/to/arma-mcp
./scripts/build-extension.sh
```

Expected generated files:

- `extension/build/ArmaMCP_x64.so`
- `extension/build/ArmaMCP_x64.dll`, when MinGW is available

The generated build directory is ignored by git.

9.1 Proton Path

For Arma 3 running under Proton:

```
Arma 3 Windows executable under Proton
  -> loads ArmaMCP_x64.dll
  -> talks to native Linux sidecar over 127.0.0.1
```

Use the Windows DLL for client-side Eden testing under Proton. The native Linux `.so` is useful for local smoke testing and future native server work, but it is not the first Proton client path.

10 Build The HEMTT Addon

From the repository root:

```
cd /path/to/arma-mcp
hemtt build
```

For a signed local release:

```
./scripts/release-signed.sh
```

This runs `hemtt release`, checks that the PBO has a matching `.bisign`, checks that HEMTT emitted the public `.bikey`, and verifies the signature with `hemtt utils verify`.

For a development deploy:

```
hemtt dev
```

`hemtt dev` builds the addon and then tries to deploy to the local Arma 3 installation. If the Arma installation path is not writable, the build can succeed while deployment fails.

11 Configure The Arma Extension

Create `ArmaMCP.ini` next to `ArmaMCP_x64.dll` in the local mod folder:

```
@ArmaMCP/
  ArmaMCP_x64.dll
  ArmaMCP.ini
  addons/amcp_main.pbo
```

The config file format is:

For a simple local smoke test:

```
host=127.0.0.1
port=38473
token=dev-token
```

The important rule is that the extension config file and sidecar use the same token. Environment variables `ARMA_MCP_HOST`, `ARMA_MCP_PORT`, and `ARMA_MCP_TOKEN` remain available as fallback settings when `ArmaMCP.ini` is absent.

12 Initial Eden Activation Test

1. Build the sidecar with `npm run build`.
2. Configure Codex MCP with the `arma-mcp` sidecar.
3. Build the native extension with `./scripts/build-extension.sh`.
4. Build the addon with `hemtt build` or deploy with `hemtt dev`.
5. Make `ArmaMCP_x64.dll` available where Arma can load it.
6. Create `ArmaMCP.ini` beside the DLL with the same token as the sidecar.
7. Launch Arma with the addon loaded and open Eden.
8. Place a simple object or game logic and select it.
9. In Codex, call `arma_ping`.
10. Call `arma_request_editor_snapshot`.
11. Wait for Eden to poll, then call `arma_get_editor_snapshot`.
12. Call `arma_generate_checkpoint_plan`.
13. Review the returned dry-run JSON.
14. Set `dryRun=false` only after review, then call `arma_queue_apply_plan`.
15. Call `arma_get_bridge_events` and inspect the result.
16. Confirm Eden creates objects around the selected anchor.
17. Press undo in Eden to confirm the operation rolls back if `collect3DENHistory` grouped changes successfully.

13 Available MCP Tools

The current sidecar exposes dotted MCP tools for bridge diagnostics, Eden reads and writes, catalog work, camera/screenshot capture, data-only composition helpers, terrain sampling, and procedural generators. Useful starting tools are:

- `arma.ping`: check MCP stdio process health without waiting on Eden.
- `arma.bridge.diagnostics`: report stdio mode, listener ownership, bridge reachability, recent bridge activity, and recent bridge errors.
- `arma.bridge.get_status`: check pending actions/results and the latest Eden bridge contact.

- `arma.bridge.get_capabilities`: inspect addon/runtime capabilities.
- `arma.eden.get_status`, `arma.eden.get_selection`, and `arma.eden.list_entities`: inspect live Eden state before planning.
- `arma.eden.batch`, `arma.eden.validate_plan`, and `arma.eden.apply_composition`: dry-run and apply reviewed changes.
- `arma.catalog.scanStart`, `arma.catalog.scanStatus`, and `arma.catalog.scanPoll`: start, inspect, and advance bounded catalog scan work.
- `arma.catalog.scanCancel`: cancel a running catalog scan.
- `arma.catalog.scanFinalize`: finish a completed bounded scan.
- `arma.catalog.scanRepair`: repair stale scan bookkeeping.

Legacy MVP aliases remain available for compatibility:

- `arma_ping`
- `arma_request_editor_snapshot`
- `arma_get_editor_snapshot`
- `arma_generate_checkpoint_plan`
- `arma_queue_apply_plan`
- `arma_get_bridge_events`

14 Troubleshooting

14.1 Codex Does Not List `arma-mcp`

- Confirm `sidecar/dist/index.js` exists.
- Confirm the Codex MCP config path is absolute and points to this repo.
- Confirm `ARMA_MCP_TOKEN` is set in the MCP server environment.
- Check sidecar stderr logs.

14.2 `arma_ping` Works But Eden Is Not Connected

- Run `arma.bridge.diagnostics` to confirm whether Codex stdio is reusing an existing bridge or owns the HTTP listener.
- Confirm Arma is running with the addon loaded.
- Confirm the native extension is available to Arma as `ArmaMCP_x64.dll` for Proton/Windows.
- Confirm `ArmaMCP.ini` sits beside `ArmaMCP_x64.dll`.
- Confirm `ArmaMCP.ini` uses the same token as the sidecar.
- Confirm the sidecar is listening on `127.0.0.1:38473`.

14.3 HEMTT Build Or Deploy Fails

- Use `hemtt build` to validate config and SQF compilation.
- Use `hemtt dev` only when the Arma installation path is writable.
- Deployment failures can happen after a successful PBO build.

14.4 Extension Build Fails

- Confirm `cmake` and a C++17 compiler are installed.
- Install `mingw-w64` if `ArmaMCP_x64.dll` is needed locally.
- The Linux `.so` can build even if the Windows cross-compiler is missing.

14.5 Plan Queue Is Rejected

- `arma_queue_apply_plan` rejects `dryRun=true`.
- Unsupported operation types are rejected.
- The MVP accepts at most 50 operations.
- Only `createObject` and `createMarker` are accepted.

15 Known MVP Limits

- Sidecar state is in-memory only.
- The addon assumes current Arma 3 support for `fromJSON` and `toJSON`.
- The extension reads token, host, and port from `ArmaMCP.ini`, with environment variables as fallback settings.
- No asset index, mesh parsing, collision validation, road detection, terrain sampling, or faction-aware object catalog exists yet.
- Marker creation still needs live Eden verification.
- BattleEye/public-server deployment, audit logs, and admin authority controls are future work.

16 One-Command Validation

After local prerequisites are installed:

```
cd /path/to/arma-mcp
./scripts/validate.sh
```

This runs sidecar validation, extension build, HEMTT build, signed HEMTT release verification, and git proof commands.